Microservices Design Patterns — Interview Edition

How to use this edition

The night-before version of the 12-pattern handbook: every pattern, the facts interviewers probe, nothing padded. Each pattern section follows the same order — **problem** → **how it works** → **trade-offs** → **when not to use** → **real examples** → **interview Q&A** → **gotchas** → **TL;DR**.

Headings: **Must-know** (problem + mechanism) · **Key facts & numbers** (trade-offs, values, when not to use, real examples) · **Code to remember** · **In the interview** (Q → short A) · **Gotchas** (mistake → correct idea) · **TL;DR**.

Pass 1 (30 min): framework page, selection table, every TL;DR. **Pass 2 (1–2 h):** Must-know + In the interview for each pattern. **Pass 3:** Last-minute checklist. Full explanations live in the chapter files (`../02-strangler-fig.md` … `../13-config-externalization.md`).

Top 25 questions across the book

#	Question	One-line answer	Ch
1	How do you replace a monolith safely?	Strangler Fig: facade in front, migrate capability by capability with flags, delete legacy on a committed date	2
2	What does Strangler Fig *not* solve?	Data consistency between old and new — shared DB first, then dual writes or CDC	2
3	Why an API Gateway?	One entry point for routing, aggregation, auth, rate limiting, logging; clients stop knowing topology	3
4	Rate-limit across several gateway instances?	Token/leaky bucket in a shared store (Redis), not in memory	3
5	API Gateway vs BFF?	One shared generic entry vs one tailored backend per client type; often combined	3–4
6	Should a BFF own a database?	No — it's a stateless projection layer	4
7	Client-side vs server-side discovery?	Client looks up + balances (Eureka+Ribbon) vs router does it (Kubernetes DNS)	5
8	Why is Eureka AP?	A stale instance list is survivable by retry; a silent registry stops everything	5
9	Registry/config server is a SPOF — so?	Run HA + cache last-known-good on clients so outages degrade, not kill	5, 13
10	Circuit breaker states?	Closed (flow, count) → Open (reject fast) → Half-Open (few probes) → Closed/Open	6
11	Why is slow worse than down?	Slow holds caller threads until timeout; enable slow-call detection	6
12	Hidden risk of a breaker?	It hides errors — alert on the Open transition	6
13	Bulkhead vs circuit breaker?	Bulkhead caps *how much* damage (always on); breaker caps *how long* you try (reactive)	7
14	How do you size a bulkhead?	Little's Law: concurrency = throughput × latency, capped by what the dependency handles	7
15	What should you retry?	Transient errors only (timeouts, 5xx, throttling) — never 400/401	8
16	Why backoff *and* jitter?	Backoff gives room to recover; jitter breaks the thundering herd	8
17	Retry a payment safely?	Idempotency key generated once, sent on every attempt; server dedupes	8
18	Why retry at one layer only?	3 layers × 3 attempts = 27 requests — a retry storm	8
19	Sidecar vs shared library?	Sidecar: language-agnostic, upgrade by image, costs a process per instance	9
20	How do you do a transaction across services?	Saga: local transactions + compensating transactions, eventual consistency	10
21	Choreography vs orchestration?	Events with no coordinator for simple flows; central orchestrator for complex ones	10
22	Saga vs 2PC?	2PC: strong but locks and blocks on coordinator failure; saga: available, eventual, manual compensation	10
23	How do you publish events reliably?	Transactional outbox; idempotent at-least-once consumers; DLQ	10–11
24	Event ordering in Kafka?	Only within a partition — key by entity ID	11

#	Question	One-line answer	Ch
25	Does CQRS need two DBs / event sourcing?	No — start with two models on one DB; the rest are separate, costly decisions	12

The microservices interview framework

Skeleton for any "design X with microservices" or "make this resilient" question. Name the neighbouring patterns — they compose.

1. **Clarify the pain.** Which problem is real — migration, many clients, unstable addresses, cascading failures, cross-service consistency, config churn? Only add a pattern for a real problem.
2. **Split and own data.** Services own their databases (database per service) → no cross-service ACID; say out loud that you accept **eventual consistency** where the business can tolerate the window.
3. **Edge / traffic story.** API Gateway (thin: TLS, coarse auth, rate limiting, correlation ID) → BFF per client type if needs diverge → Service Discovery (Kubernetes DNS or Eureka) → Sidecar/mesh for east-west concerns.
4. **Resilience story.** Every remote call gets a **timeout**; **Retry** (transient only, backoff + jitter, one layer, idempotency keys) inside a **Circuit Breaker** (rate over a window, slow calls count, local fallback), inside a **Bulkhead** (bounded pool per dependency).
5. **Data story. Saga** for multi-service business transactions (orchestrate when complex, outbox for events) → **EDA** for decoupled reactions (at-least-once, partition keys, DLQ) → **CQRS** only where read/write needs truly diverge.
6. **Operations. Config Externalization** (Git-backed, refresh without restart, secrets in Vault), correlation IDs and distributed tracing, alerts on breaker state and retries.
7. **Migration.** If there's a legacy system, **Strangler Fig** with shadow mode and a decommission date.
8. **Close with trade-offs.** Every centralised component (gateway, registry, config server, orchestrator, mediator) is a SPOF → HA + client caching. Every indirection (gateway, BFF, sidecar, discovery) is a network hop and a new failure mode.

Pattern-selection table

Symptom you hear	Reach for	Pair it with	Don't use when
"We must replace the legacy monolith"	Strangler Fig	API Gateway as facade, feature flags in external config	Nothing in use needs replacing
"Clients call many services / many round trips"	API Gateway	Discovery, circuit breaker per route	Few services; internal-only traffic (mesh)
"Mobile/TV/web need different shapes and logic"	BFF	Thin shared gateway, shared libs, contract tests	Clients' needs are nearly identical
"IPs change; autoscaling; containers"	Service Discovery	Retry on another instance, client caching	On Kubernetes, adding Eureka/Consul
"One slow dependency takes everything down"	Circuit Breaker	Timeouts, retry, bulkhead, fallback	In-memory calls unlikely to fail
"Failure in one area starves unrelated work"	Bulkhead	Circuit breaker, timeouts, bounded queues	As a scaling tool
"Occasional blips, timeouts, throttling"	Retry	Circuit breaker, idempotency keys, overall deadline	Permanent errors, long outages, strict latency SLA
"Same logging/mTLS/metrics in 4 languages"	Sidecar (mesh)	Discovery, CB/retry in the proxy	Few services in one language
"Order + payment + inventory must all succeed or undo"	Saga	EDA, outbox, idempotency, retry	A single DB where local ACID works
"Many reactions to one action; loose coupling"	Event-Driven	Saga, CQRS, DLQ, correlation IDs	You need an immediate answer
"Reads dwarf writes; one model can't serve both"	CQRS	EDA, event sourcing (maybe), saga	Simple CRUD
"Config change needs a redeploy; secrets everywhere"	Config Externalization	Discovery to find the server, Vault for secrets	Env vars/ConfigMaps already suffice

2. Strangler Fig

Replace a legacy system gradually behind a routing facade instead of a big-bang rewrite. Full notes: 02-strangler-fig.md

Must-know

- **Problem** — legacy systems are painful to live with and to replace; a **big-bang rewrite** chases a moving target and discovers all undocumented behaviour on switchover day.
- **How it works** — build the new system alongside the old; put a **facade/router** in front of both; migrate **one capability at a time**; when no traffic reaches legacy, **delete it**. Named after a tree that grows around its host until the host rots away.
- **Facade** — every request hits it first and it asks one question: *has this functionality been migrated yet?* Clients see one address and never know there are two systems.
- **Routing logic** — **feature flags** (instant rollback, no deploy), **user segmentation** (employees → 1% → 10% → all, or one small market first), **API versioning** (`/v1/orders` → legacy, `/v2/orders` → new).
- **Consistent routing** — hash a **stable user ID**, never `random()` per request, so one user always gets the same system.
- **Shadow mode** — send the request to both, return the legacy answer, log mismatches; finds undocumented behaviour before users see it.

- **Pick the seam first** — "cleanly isolated" means no shared mutable state; look for read-heavy capabilities or tables nothing else writes.

Key facts & numbers

Trade-off	What it costs
Two systems in parallel	Operate, monitor, pay for both, sometimes for years (two pipelines, two on-calls)
Facade	Single point of failure on **every** request; needs HA and a latency budget
Extra hop	Adds latency to every request
Data consistency	The pattern gives no help; hardest part of real migrations
Duplicated logic	Same rule lives in two places while a capability is in flight

Data strategy (easiest → hardest)	Note
Shared database	Start here; no sync problem; split data later as its own migration
Dual writes	New system writes both; no transaction spans both → divergence if one write fails
CDC (Debezium on the transaction log → Kafka)	New side eventually consistent; **legacy needs no change**

- **When not to use** — nothing in use needs replacing, or you cannot put a routing layer in front of the old system. (It works for any in-use component: payment provider, search engine, cloud move — not just ancient monoliths.)
- **Real examples** — swapping a payment provider, replacing a search engine, migrating clouds; the facade frequently becomes the permanent **API Gateway**.

In the interview

Q: Why Strangler Fig over a rewrite? **A:** Every step is small, tested in production, and reversible in seconds; you prioritise by business pain, get early feedback, and avoid a catastrophic cutover date.

Q: Why route by a hash of user ID rather than randomly? **A:** A user must see consistent behaviour; random routing turns any difference into an unreproducible intermittent bug.

Q: What does the pattern *not* solve? **A:** Data consistency between old and new; usually keep a shared DB, then split it later (or use CDC).

Q: Do you migrate the database first? **A:** Usually the opposite — move behaviour first on the shared DB, split data later because it is the riskiest part.

Q: Biggest real-world failure mode? **A:** Migration fatigue: it is safe to stop at 60%, so teams run both systems forever. Set a decommission date up front as a real deliverable.

Gotchas

- "New system must be microservices" → it is about *how* you migrate; you can strangle a monolith into a better monolith.
- Deleting legacy code the moment traffic hits zero → keep it unreachable but intact for a full business cycle; it is your rollback.
- Deploying facade + first migration in one release → you cannot tell which change broke things.
- Flags that outlive the migration → schedule flag removal as part of "done".

TL;DR

Facade in front, migrate capability by capability with flags and consistent routing, shadow-compare, delete legacy on a committed date. Data is the part you must solve yourself.

3. API Gateway

A single entry point in front of all microservices for external clients. Full notes: 03-api-gateway.md

Must-know

- **Problem** — many services, many clients: each client must know every service's location, auth and response shape; many round trips per screen; topology frozen into apps you cannot force-update (TV, mobile).
- **How it works** — clients make **one call**; the gateway **routes**, **aggregates** responses, and returns **one unified response**, hiding service topology.
- **Responsibilities** — routing, aggregation, authentication **and** authorization (two separate checks), rate limiting, load balancing, caching/compression, logging/monitoring, request transformation, version management, partial-failure handling.
- **Scatter-gather** — one request fans out to several **parallel** service calls and returns merged (Instagram feed = Feed + Notification services).
- **Why fan-out wins** — one phone round trip + parallel calls inside the data centre beats several sequential round trips over a mobile network.
- **Transport, not domain** — keep business rules out of the gateway.

Key facts & numbers

Problem the gateway introduces	Mitigation
Single point of failure	HA + redundancy (≥2 replicas behind an LB), monitor closely
Bottleneck	Scale horizontally, optimise code
Accumulating complexity	Keep responsibilities clear; split per client type or per domain
Maintenance drift	Automate routes/rules via CI/CD
Prime attack target	Harden, patch, monitor
Extra network hop	Caching, response compression
Distributed rate limiting	**Token bucket / leaky bucket** with a **shared store (Redis)**
Aggregation cost	Async, non-blocking fan-out

	API Gateway	BFF
Count	One shared	One per client type
Optimised for	All clients generically	One client
Main risk	Bottleneck / distributed monolith	Duplication and drift

- **When to use** — multiple services *and* multiple client types; many round trips per screen; want auth/rate limiting/logging in one place; exposing to untrusted external consumers.
- **When not to use** — very few services; purely internal service-to-service traffic (use a **service mesh**); one client with sharply different needs (use BFF); ultra-low-latency paths where a hop is unacceptable.
- **Real examples** — Netflix (TV app makes one call instead of calling preferences, subtitles, catalogue…); Instagram (upload = sequential auth→route; feed = parallel fan-out + aggregate); Spring Cloud Gateway in production.

Code to remember

```
// Parallel fan-out with a per-call timeout (not .get() as you go = sequential)
CompletableFuture<Feed> f = supplyAsync(() -> feedSvc.get(user));
CompletableFuture<Notifs> n = supplyAsync(() -> notifSvc.get(user));
Feed feed = f.get(1, TimeUnit.SECONDS);
Notifs notifs = n.get(1, TimeUnit.SECONDS);
return new FeedPage(feed, notifs);
```

In the interview

Q: Isn't it a single point of failure? **A:** Yes by design; run multiple replicas behind a load balancer, monitor it like any critical service.

Q: How do you rate-limit across 3 gateway instances? **A:** Token/leaky bucket backed by a shared store (Redis); in-memory limits of 100 req/s each actually allow 300.

Q: Gateway vs service mesh? **A:** Gateway handles north-south external traffic; mesh (sidecars) handles internal east-west traffic.

Q: Biggest long-term risk? **A:** Business logic creeping in until every team must change it — a distributed monolith and deployment bottleneck.

Gotchas

- Sequential calls that look concurrent (calling `.get()` on each future as created) → start all, then join.
- No per-call timeout in aggregation → the slowest dependency dictates latency.
- Retrying POSTs at the gateway → duplicate orders; retry only idempotent methods (e.g. GET).
- Not generating a **correlation ID** at the gateway → no cross-service tracing.

TL;DR

One front door for routing, aggregation and cross-cutting concerns; run it HA, fan out in parallel with timeouts, keep it about transport.

4. Backends for Frontends (BFF)

One tailored backend per client type (web, mobile, TV) instead of one shared API. Full notes: 04-bff.md

Must-know

- **Problem** — a one-size-fits-all backend is shaped by the biggest/oldest client (web); mobile gets **over-fetching** (more data than needed) and many round trips on slow, metered networks.
- **How it works** — each client type gets its own BFF; every BFF calls the **same** microservices, then **aggregates and transforms** for its client (mobile: small images, short text; TV: hi-res, full cast).
- **Projection layer** — BFFs are **stateless**; microservices remain the source of truth.
- **Ownership** — usually owned by the **frontend team**, so they ship independently; scale per client.
- **Origin** — coined by **Sam Newman**.
- **Per client type, not per screen** — all mobile screens share the Mobile BFF.

Key facts & numbers

Cost	Mitigation
Code duplication (auth, errors, logging)	Shared libraries (`bff-common`), internal shared services
More codebases/pipelines	DevOps automation, monitoring
Frontend↔BFF coordination	Clear contracts + **consumer-driven contract testing**
Extra hop	Place BFFs close to their frontends; efficient protocols
Same data cached in several shapes	Caching at BFF + frontend, `Cache-Control`
More load on shared services	Scale backends, rate-limit

Netflix BFF	Serves
Mobile	Concise details, smaller images, subset of recommendations
TV	Rich visuals, full details; **extra HD-entitlement authorization check**
Web	Balance of performance and richness

- **When not to use** — clients' needs are nearly identical (a shared gateway is simpler); team too small to maintain several backends. Adopt it when you start adding device-conditionals to shared code.
- **Real examples** — Netflix (phones, TVs, browsers); e-commerce product page (Web BFF detailed view vs Mobile BFF concise view of the same product).

In the interview

Q: Strongest argument over a shared gateway? **A:** Client-specific logic like the TV HD-entitlement check lives in its own BFF instead of polluting shared code with device conditionals.

Q: Should a BFF own a database? **A:** No — it is a projection layer; a BFF with its own DB has accidentally become a microservice with duplicated data ownership.

Q: Doesn't GraphQL make BFF obsolete? **A:** It solves response *shaping*, not client-specific *logic*, independent scaling or ownership; some teams run a GraphQL layer per client type.

Q: Does BFF make things faster? **A:** It adds a hop, but is faster *for the client* by cutting round trips and payload size, which dominates on mobile.

Gotchas

- Sequential upstream calls in the BFF → just moves the latency; fan out with `CompletableFuture.allOf`.
- No fallback for secondary data → reviews down should not break the product page; only the primary resource may fail the request.
- Business rules ("is this refundable?") in the BFF → belong in the owning service; BFF holds **presentation** decisions only.
- Breaking changes → old mobile versions live for years; version paths (`/mobile/v1` , `/mobile/v2`).

TL;DR

Per-client backend that shapes shared data for one frontend, owned by that frontend's team; stateless, fan out in parallel, share common code.

5. Service Discovery

Find services by logical name when instance addresses change constantly. Full notes: [05-service-discovery.md]

Must-know

- **Problem** — containers and auto-scaling give instances new IPs constantly and an unknown instance count; hardcoded endpoints couple every client to every deployment. (A monolith doesn't need it:

calls are in-process.)

- **How it works** — instances **register** with a **service registry** (name, IP, port, metadata) at startup, **deregister** on shutdown, and **health checks** remove crashed ones; clients look up by **logical name**, so addresses can change freely.
- **Client-side discovery** ("smart client") — client queries the registry and load-balances itself (**Eureka + Ribbon / Spring Cloud LoadBalancer**).
- **Server-side discovery** ("smart router, dumb client") — client calls a fixed endpoint (LB, gateway, DNS name, virtual IP) that does the lookup (**Kubernetes Services**, cloud LBs). The norm in modern systems.
- **Self- vs third-party registration** — service registers itself (simple, but its job to stay current) vs a service manager registers it (another component to run).
- **Health models** — **heartbeat** (service pushes; load on services) vs **active monitoring** (registry polls; load on registry).
- **Discovery ≠ load balancing** — discovery says *what instances exist*; LB picks *which one*.
- **Register only when ready, deregister before going offline** — gate with a **readiness** probe, not liveness.

Key facts & numbers

	Client-side	Server-side
Who queries registry	Client	LB / gateway
Hops	Fewer (direct call)	One more
Client complexity	High (library per language)	Very low
Polyglot	Poor	**Excellent** (any HTTP client works)
Routing control	Fine-grained, app-specific	Whatever the router does
Typical stack	Eureka + Ribbon	Kubernetes, cloud LBs

Registry	CAP choice	On partition
Zookeeper / Consul	**CP**	May refuse to answer until consensus
Eureka	**AP**	**Self-preservation**: keeps serving (possibly stale) entries

- Clients **cache** the instance list (refresh ~every 30 s) — cuts latency and survives registry outages.
- Heartbeat TTL: **10 s heartbeat / 30 s expiry** is a reasonable start; a 90 s TTL = up to 90 s of traffic to a dead instance.
- Kubernetes: Service = stable virtual IP + DNS name; **etcd**-backed registry + **CoreDNS**; kube-proxy balances to current pods.
- **When not to use** — a monolith; or adding Eureka/Consul **on Kubernetes** (two competing sources of truth).
- **Real examples** — Netflix Eureka; Kubernetes (most widespread); Airbnb **SmartStack** (HAProxy per host + ZooKeeper); Consul (Stripe, PagerDuty); Lyft **Envoy** sidecars fed by a control plane.

In the interview

Q: Client-side vs server-side discovery? **A:** Who does lookup + LB: client (no extra hop, smart routing, library per language) vs router (simple, language-agnostic, extra hop, router must be HA).

Q: Isn't the registry a SPOF? **A:** Yes — cluster it and cache last-known-good on clients so an outage degrades (stale data) rather than stops you.

Q: Why is Eureka AP? **A:** A stale list is survivable (call fails, retry another instance); a registry that won't answer stops everything.

Q: Why not just DNS? **A:** Clients and runtimes cache DNS aggressively, sometimes forever; Kubernetes pairs DNS with a stable virtual IP instead of raw pod IPs.

Q: Intermittent errors right after deploys? **A:** Instances registering before warm-up or not deregistering before shutdown.

Q: Security risk? **A:** Service spoofing — anyone who can write to the registry can register a fake instance and intercept traffic; use ACLs/tokens or mTLS, keep it internal, TLS all traffic.

Gotchas

- Name → single address map → that's a naming service, not discovery: no LB, no failover.
- Discovery is not a substitute for retry → it knows what *was* alive; retry on another instance.

TL;DR

Registry + logical names + health checks; client-side (Eureka) vs server-side (Kubernetes DNS); HA registry, client caching, register-when-ready.

6. Circuit Breaker

Stop calling a dependency that is already failing, so failure doesn't cascade. Full notes: 06-circuit-breaker.md

Must-know

- **Problem — cascading failure**: B hangs → A's threads block waiting → A runs out of threads and dies though its code is fine. **Slow is worse than down** (dead refuses instantly; slow holds threads hostage).
- **How it works** — wrap each remote call; count outcomes; once failures cross a threshold, **reject instantly** with a fallback; after a timeout, **probe** with a few calls.
- **CLOSED** = traffic flows, failures counted (electrical: a closed loop conducts). **OPEN** = calls blocked immediately. **HALF-OPEN** = a few trial calls: success → Closed, failure → Open and the timer restarts.
- **Two benefits of Open** — the caller stops wasting threads, **and** the failing service stops receiving load so it can recover.
- **Fallback** — cached data, default value, simpler alternative, or queue for later → **graceful degradation**.
- **One breaker per dependency**, never one per application.

Key facts & numbers

Parameter (Resilience4j example)	Meaning
`slidingWindowSize: 20`	Judge the last 20 calls (rate over a window, not a raw count)
`minimumNumberOfCalls: 10`	Don't judge until 10 samples
`failureRateThreshold: 50`	Open if >50% failed
`waitDurationInOpenState: 10s`	How long to stay Open
`permittedNumberOfCallsInHalfOpenState: 3`	Trial calls in Half-Open
`slowCallDurationThreshold: 2s` + `slowCallRateThreshold: 50`	Treat slow calls as failures
`ignoreExceptions`	Business errors (400/validation) must not trip it

Tuning	Too low	Too high
Failure threshold	Trips on glitches, **flapping** (1 failure = too sensitive)	Protects too late (100 = too late)
Open timeout	Probes before recovery → open/half-open flapping	Refuses calls after dependency is healthy

- **When not to use** — in-memory operations unlikely to fail; very localised, self-correcting errors where an aggressive breaker trips constantly.
- **Real examples** — **Netflix Hystrix**: recommendations down, video still streams (row omitted); product page skips reviews ("Reviews unavailable"); DB reads redirected to replica/cache; one breaker per external weather API; service meshes and gateways per route. Libraries: **Resilience4j** (Java standard; Hystrix in maintenance), **Polly** (.NET).

Code to remember

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResult charge(Order order) { return paymentClient.post(order); }

// same signature + trailing Throwable; must be local and fast
private PaymentResult paymentFallback(Order order, Throwable t) {
    return PaymentResult.deferred(order.getId());   // queue for later
}
```

In the interview

Q: Explain the three states. **A:** Closed counts failures while traffic flows; Open rejects instantly after the threshold; Half-Open lets a few probes through after a timeout — success closes, failure reopens.

Q: Why only a few requests in Half-Open? **A:** To avoid re-killing a service that has only just started recovering.

Q: Why is slow worse than down? **A:** A slow dependency holds caller threads until timeout, exhausting the pool; so enable slow-call detection.

Q: Timeouts vs circuit breaker? **A:** Timeouts bound each wait; the breaker prevents waiting at all once a service is known bad. Need both — the breaker only sees calls that return.

Q: Breaker vs retry? **A:** Opposites used together: retry tries harder on transient faults; the breaker gives up sooner on real outages and stops retries becoming an attack.

Q: Hidden risk of a breaker working well? **A:** It suppresses the errors that would raise the alarm — alert on the Open transition (e.g. `resilience4j_circuitbreaker_state`).

Gotchas

- "Open means working" → Open = blocked, Closed = flowing.
- Counting 400s/validation errors as failures → trips on a client bug and takes down a healthy dependency.
- Fallback that makes another network call → a second failure waiting to happen; keep it local (cache, default, queue).

TL;DR

Closed → Open → Half-Open per dependency, rate-based over a window, slow calls count, fallback locally, alert on Open.

7. Bulkhead

Partition resources so one failing dependency cannot starve everything else. Full notes: 07-bulkhead.md

Must-know

- **Problem** — failure propagation: Order Service calls Inventory and Shipping from **one shared thread pool**; Inventory gets slow, every thread blocks on it, and Shipping calls fail though Shipping is healthy.
- **How it works** — like a ship's **watertight compartments**: give each dependency/workload its **own bounded pool** (threads, connections, semaphores, CPU/memory), so a failure is contained to its compartment.
- **Isolate resources, not just code** — two modules separate in source can still kill each other through a shared pool (the half people miss).
- **Two levels** — *between* services (user management vs streaming) and *within* one service (song-fetch pool vs song-stream pool, so playback continues when the DB is slow).
- **Resilience, not scalability** — more threads = more context switching = *less* throughput; capacity comes from horizontal scaling or async.

Key facts & numbers

Resilience4j type	Mechanism	Use when
`SemaphoreBulkhead`	Caps **concurrent calls** on caller's thread (e.g. `maxConcurrentCalls: 25`, `maxWaitDuration: 100ms`)	Default — cheap, no extra threads
`ThreadPoolBulkhead`	Runs calls on a **separate pool** (`maxThreadPoolSize`, `coreThreadPoolSize`, `queueCapacity`)	True thread isolation or async

- **Sizing — Little's Law**: `concurrency = throughput × latency`. 50 req/s × 0.2 s = **10** in flight → size a bit above (~15). Never above what the dependency can handle (a pool of 500 for a dependency that degrades past 100 just moves the queue into them).
- Short `maxWaitDuration` (~100 ms) → **shed load** rather than accumulate it.
- Other bulkheads: Kubernetes pod `requests` / `limits`; **DB connection pool** (e.g. Hikari `maximum-pool-size: 20`, `connection-timeout: 3000`) — often the scarcest resource.
- Each thread ≈ **1 MB stack**: 20 pools × 50 threads ≈ 1,000 threads ≈ 1 GB before serving anything.

	Bulkhead	Circuit Breaker
Mechanism	Partitions resources	Stops calls after repeated failures
When it acts	Always (structural)	On detected failures (reactive)
Analogy	Watertight compartments	A fuse that trips
Limits	**How much** damage	**How long** you keep trying

- **When not to use / limits** — not a load or scaling tool; not a silver bullet (a total network outage still takes you down); too many tiny pools waste memory — bulkhead by *dependency*, not by endpoint.
- **Real examples** — e-commerce (search spike must not slow cart/orders); gaming (one game per service); finance (**slow reporting must not slow transactions**); Spotify-like streaming (users keep listening while user-management is overloaded).

Code to remember

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(50),              // bounded queue
    new ThreadPoolExecutor.AbortPolicy());     // reject when full
```

In the interview

Q: Why can a slow Inventory break healthy Shipping? **A:** They share one thread pool; Inventory's blocked threads consume all of it.

Q: Bulkhead vs circuit breaker? **A:** Bulkhead caps blast radius and is always on; breaker stops feeding the fire once a dependency is known bad. Use both.

Q: Bulkhead vs rate limiting? **A:** Rate limiting caps requests you *accept*; bulkheads cap resources a category of work can *consume*.

Q: How do you size a pool? **A:** Little's Law as a start, capped by what the dependency can handle, then tune with metrics.

Q: Need microservices for it? **A:** No — separate thread pools per workload inside a monolith is often the highest-value start.

Gotchas

- `Executors.newFixedThreadPool` uses an **unbounded** queue → trades thread exhaustion for `OutOfMemoryError`.
- Bulkhead without timeouts → pool full of threads blocked forever; bulkhead limits *how many*, timeout limits *how long*.
- Bulkheading HTTP calls but sharing one DB connection pool → one slow query starves every endpoint.

TL;DR

Bounded pool per dependency (threads, connections, pods), sized by Little's Law, with timeouts and load shedding; it contains failure, it doesn't add capacity.

8. Retry

Ride out transient faults by re-attempting with backoff and jitter. Full notes: 08-retry.md

Must-know

- **Problem** — networks and services have **transient** faults (dropped packets, brief connection exhaustion, LB restarts) that fix themselves in seconds; failing immediately turns a blip into an error.
- **How it works** — wrap the call in a **policy**: max attempts, which errors are retryable, delay strategy; the invoker retries until success or gives up and propagates the final failure.
- **Transient vs permanent** — retry timeouts, 5xx/502, deadlocks, connection refused, throttling; **never** 400, 401, business/config errors.
- **Exponential backoff** — 1 s, 2 s, 4 s, 8 s… **capped**; don't hammer a service that may be failing because it's overloaded.
- **Jitter** — random variation in the delay (full / equal / decorrelated) to break the **thundering herd** (10,000 clients retrying in unison re-kill a just-recovered service).
- **Idempotency** — the client cannot tell a lost request from a lost response, so the server must dedupe with an **idempotency key** (Stripe's API assumes clients will retry).

Key facts & numbers

Backoff	Behaviour	When
Constant	Same wait (e.g. 5 s)	Predictable recovery
Exponential	1 s, 2 s, 4 s…	General default
Capped exponential	As above with a max	Prevents absurd waits

Number	Why it matters
3 layers × 3 attempts	1 request → **27** requests (A→B→C with 2 retrying layers → **9**)
Uncapped: base 1 s, ×2, 10 attempts	Final wait ≈ **17 minutes**
5 s timeout × 3 attempts + backoff	≈ **20 s** worst case → needs a time limiter (e.g. 3 s)
Resilience4j `maxAttempts: 3`	= 1 initial + 2 retries

	Retry	Fail-fast (Circuit Breaker)
Favours	Eventual success	Conserving resources
Right for	Transient faults	Lasting / permanent faults

- Order: Resilience4j default is **Retry(CircuitBreaker(call))** — breaker sees each attempt; inverted, it counts a whole retry sequence as one failure and opens too slowly.
- **When not to use** — non-transient errors; dependency known to be down for long (planned outage); strict latency SLAs; very expensive or unrepeatable side effects → use compensation, reconciliation, or alert a human.
- **Real examples** — AWS/Google Cloud SDKs (built-in backoff + jitter); Netflix retries on **another instance**; Stripe/PayPal idempotency keys; RabbitMQ/Kafka requeue and **dead-letter queues**; inventory lock timeout retried a minute later.

Code to remember

```
String key = UUID.randomUUID().toString();       // ONCE, outside the retry loop
executeWithRetry(() -> http.post("/payments")
    .header("Idempotency-Key", key)               // same key on every attempt
    .body(payment).execute(), 3, 200, 2.0, true);
// Server: unique index on key (Redis w/ TTL or table); replay stored response if seen
```

In the interview

Q: What should you retry? **A:** Only transient errors (timeouts, 5xx, throttling, connection refused); let 4xx and business errors fail fast.

Q: Why backoff *and* jitter? **A:** Backoff gives a struggling service room; jitter de-synchronises clients so retries don't arrive as one wall of traffic.

Q: How do you retry a payment safely? **A:** Idempotency key generated once per logical operation, sent on every attempt; server dedupes via a unique constraint and replays the first response.

Q: Why retry at only one layer? **A:** Layered retries multiply (3 layers × 3 = 27 requests) into a retry storm against an already struggling service.

Q: Retry vs circuit breaker? **A:** Complements: retry for hiccups, breaker for outages; the breaker stops retries hammering a downed service.

Q: Why an overall timeout? **A:** Individual timeouts × attempts + backoff silently blow the latency budget.

Gotchas

- Key generated inside the retry loop → every attempt looks new → triple charge.
- Catching broad `Exception` → retries permanent errors too.
- "It's a GET so it's idempotent" → check the whole path (a GET that bumps a counter isn't).
- Retries not logged/metered → a dependency drifting from 0.1% to 15% failure looks healthy until it tips over.

TL;DR

Transient-only, capped exponential backoff + jitter, few attempts, overall deadline, idempotency keys, one layer, paired with a circuit breaker.

9. Sidecar

Move cross-cutting concerns into a co-located helper process next to each service. Full notes: 09-sidecar.md

Must-know

- **Problem** — every service needs logging, metrics, auth, mTLS, retries, discovery; a shared library must be rewritten **per language** in a polyglot estate and the versions drift; upgrades mean redeploying every service.
- **How it works** — deploy a helper as a **separate process in its own container**, **co-located in the same Kubernetes Pod**: shared network namespace (talk over `localhost`), shared volumes, **shared lifecycle** (created and destroyed together).
- **Proxy role** — the sidecar **transparently intercepts** all inbound/outbound traffic, so it can do load balancing, circuit breaking, mTLS and discovery **without a line of app code changing**. "Intercepts", not "calls", is the key word.
- **Service mesh** — a sidecar proxy next to every service + a central **control plane** configuring them (**Istio + Envoy**).
- **Correction to the course** — not "same process/same container": separate container, same Pod; same-process would kill isolation and language freedom.

Key facts & numbers

	Shared library	Sidecar
Languages	One implementation per language	One, language-agnostic
Upgrade	Rebuild + redeploy every service	Update the sidecar image
Runtime cost	Negligible	An extra process **per instance**
Failure isolation	Library bug crashes the app	Can fail separately
Best when	One language, few services, latency-critical	Polyglot, many services, frequent policy changes

- Cost at scale: **100 MB × 1,000 pods = 100 GB RAM** of infrastructure — the main objection to a mesh.
- Considerations: version the sidecar against the app; resource hogging; wider **attack surface** (holds mTLS keys, reads app output); more moving parts to debug.
- **When not to use** — below roughly a dozen services in one or two languages (a library is cheaper); ultra-low-latency paths (measure the hop).
- **Real examples** — **Istio/Envoy** (all networking, policy, telemetry in the sidecar; app has zero networking code); Lyft built Envoy; log forwarders and metrics collectors.

In the interview

Q: Why not a shared library? **A:** Library = one per language, drifting, redeploy-to-upgrade; sidecar is language-agnostic and upgraded by image. Library wins for one language, few services, latency-critical paths.

Q: Is a sidecar a microservice? **A:** No — it serves exactly one app instance, owns no domain, and its lifecycle is bound to its host.

Q: Main cost of a mesh? **A:** Cumulative CPU/memory of one proxy per instance, plus operational complexity.

Q: Does mTLS solve authorization? **A:** No — it proves *who* is calling, not *what they may do*; you still need authorization policies.

Q: Gateway vs mesh? **A:** Gateway handles north-south (external) traffic; the sidecar mesh handles east-west (internal).

Gotchas

- App starts before the proxy is ready → first outbound call fails; use native sidecars or Istio `holdApplicationUntilProxyStarts: true`.
- Proxy dies before the app at shutdown → in-flight requests fail.
- Retries in both app and mesh → 3 × 3 = 9 requests; pick one layer (usually the mesh).

TL;DR

Same Pod, separate container, transparent proxy for networking/security/observability; language-agnostic but costs a process per instance.

10. Saga

Distributed transactions as a sequence of local transactions with compensations. Full notes: 10-saga.md

Must-know

- **Problem** — with **database per service** there is no shared transaction manager for one ACID transaction; holding locks across seconds of network calls would block every other user.
- **How it works** — a saga = **sequence of local transactions**; each updates its own DB and publishes an event/triggers the next; on a business-rule failure, run **compensating transactions** for earlier steps (in reverse).
- **Guarantee** — either all steps complete, or compensations undo the completed ones → **eventual consistency**, **no isolation** (others can see half-finished state).
- **Choreography** — services emit/consume domain events, no coordinator (`OrderCreated` → payment → `PaymentCompleted` → inventory…).
- **Orchestration** — a central **orchestrator** tells each participant what to do and runs compensations; the flow is readable in one place.
- **Saga log** — persisted record of every step and compensation, so a crashed saga can resume or compensate.
- **Compensation ≠ rollback** — a rollback erases history; a compensation is a **new forward action** (the refund appears next to the charge). "Cancelling an order is not the same as never placing it."
- **Transactional outbox** — write the state change and the outgoing event in **one local transaction**; a poller or Debezium CDC publishes it.

Key facts & numbers

	Choreography	Orchestration
Coordination	None	Central orchestrator
Service autonomy	Higher	Lower
Flow visibility	Poor (scattered)	Good (one place)
Error handling	Harder	Easier
Coupling	To events	To the orchestrator
Best for	**Simple** (2–3 services)	**Complex** (move past 3–4 steps)

	2PC	Saga
Consistency	Strong (ACID)	Eventual
Locks	Held across all participants until commit	No distributed locks
Availability	**Blocks** if coordinator dies mid-protocol	Stays available
Rollback	Automatic	Hand-written compensations
Suits	Short, one trust domain	Long-lived, cross-service, cross-company

- **When not to use** — a single database where one local ACID transaction works; flows that can't tolerate a window of intermediate state.
- **Real examples** — (tools: Temporal, Camunda/Zeebe, AWS Step Functions, Axon) e-commerce checkout (order → payment → inventory; refund on stock failure); **travel booking** across airline/hotel/car

companies (shared transaction impossible); food delivery (restaurant out of item → cancel + refund); bank transfer (debit savings / credit checking).

Code to remember

```
try {
    activities.createOrder(order);   saga.addCompensation(activities::cancelOrder, order.id());
    activities.chargePayment(order); saga.addCompensation(activities::refundPayment, order.id());
    activities.reserveStock(order);                                  // may fail
} catch (ActivityFailure e) {
    saga.compensate();   // registered compensations run in reverse
    throw e;
}
```

In the interview

Q: Why not one ACID transaction across services? **A:** Each service owns its DB — no shared transaction manager or log — and locks held across network calls block everyone.

Q: Choreography or orchestration? **A:** Choreography for simple 2–3-step flows; orchestration for complex ones — visibility and error handling, and the business process is written somewhere.

Q: Saga vs 2PC? **A:** 2PC is strongly consistent but holds locks and blocks if the coordinator dies, and external parties won't support it; sagas stay available with eventual consistency and manual compensations.

Q: Why must steps be idempotent? **A:** A step can succeed but fail before reporting success and be retried; guard with the saga ID. Compensations too — refunding twice is as bad as charging twice.

Q: What about steps you can't undo (email, shipping)? **A:** Put irreversible actions **last**, or design a real-world compensation.

Q: Isn't the orchestrator a SPOF? **A:** Make it HA with persisted state; choreography's risk is comprehensibility, not availability.

Gotchas

- Assuming isolation → users can see an order about to be cancelled; show "processing", not "confirmed".
- Compensation that fails permanently → retries with backoff, then dead-letter queue + human; alert on half-compensated sagas.
- In-memory saga log → a crash leaves orphaned state; persist before each step.
- No per-step timeout → a silent participant hangs the saga forever.

TL;DR

Local transactions + compensations, eventually consistent, no isolation; orchestrate complex flows, idempotent steps, outbox for events, irreversible steps last.

11. Event-Driven Architecture (EDA)

Components communicate by publishing events to a bus instead of calling each other. Full notes: 11-event-driven.md

Must-know

- **Problem** — direct synchronous calls tightly couple services; every new reaction means changing and redeploying the producer; callers block; real-time reactions are hard.
- **How it works** — a **producer** publishes a state change (an **event**, named in the **past tense** — it already happened and can't be refused) to an **event bus**; **consumers** subscribe to **channels** and react. The producer doesn't know who listens.
- **Change-cost win** — `PhotoUploaded` triggers notify followers, update profile, moderation; adding "generate thumbnail" later = **new consumer, upload code untouched**.

- **Variants** — **Broker** (producers publish straight to channels; simple, scalable, less control), **Mediator** (central component routes/filters/transforms; control but potential **bottleneck**), **Hybrid**.
- **Event vs call** — a call has one target and expects a return; an event has zero-or-many unknown targets and expects nothing. Queries needing an immediate answer stay synchronous.
- **Event sourcing** — store state as a sequence of events (`Deposited £100` , `Withdrew £30`) and derive the balance by replay; gives audit, versioning, "time-travel".
- **Delivery** — usually **at-least-once** → duplicates happen → consumers must be **idempotent**.

Key facts & numbers

Concern	Rule
Ordering	Not guaranteed system-wide; Kafka orders **within a partition** → key by entity ID
Schemas	**Adding an optional field is safe; removing/renaming breaks** old consumers; enforce with a schema registry (`BACKWARD` compatibility)
Durability	Persistence, acknowledgements, retries, **dead-letter topic**
Debugging	Causation spans services and time → **correlation IDs** + **distributed tracing** from day one

Kafka setting	Why
`acks: all`	Wait for all replicas — no silent loss
`enable.idempotence: true`	Broker de-dupes producer retries
`enable-auto-commit: false`	Commit offset only **after** successful processing
DLT + `ExponentialBackOff(1000, 2.0)`	1 s, 2 s, 4 s… then park the poison message

- **When not to use** — you need an immediate answer ("is this user authorized?"); strict global ordering; a small system where the tracing/testing overhead isn't justified.
- **Real examples** — ride-sharing (request → nearby drivers → accept → live location → completion → billing → payment); IoT/smart home; real-time analytics (stock data); e-commerce order → inventory, billing, shipping; traffic sensors adjusting signals.

In the interview

Q: What does EDA buy you? **A:** Loose coupling (add consumers without touching producers), async non-blocking producers, independent scaling, real-time reaction.

Q: Broker vs mediator? **A:** Broker: straight to channels, scalable, less control. Mediator: central routing/transformation, more control, possible bottleneck.

Q: How do you handle duplicates? **A:** Assume at-least-once; store processed event IDs and skip repeats.

Q: How do you guarantee ordering? **A:** Only per partition — key events by entity ID (e.g. account ID) so `AccountCreated` precedes `AccountDeleted` .

Q: How do you evolve event schemas? **A:** Additive changes only, via a schema registry that fails the build on a breaking change.

Q: Why is it hard to debug? **A:** No single stack trace — the cause was published minutes ago by another service; correlation IDs and tracing fix it.

Q: What should an event carry? **A:** Identifiers plus the facts that changed — too little recreates call-back coupling, too much makes it a schema everyone depends on.

Gotchas

- Auto-commit on → offset committed before the handler finishes → silent loss on crash.
- No dead-letter queue → one poison message blocks its partition forever.
- Synchronous in-memory `publish` calling consumers on the producer's thread → kills the asynchrony that is the point.

- Events named as commands (`UpdateEmail` vs `EmailChanged`) → disguised instructions, reintroduced coupling.

TL;DR

Publish past-tense facts to a bus; consumers react independently. Plan for at-least-once, per-partition ordering, additive schemas, correlation IDs and DLQs.

12. CQRS — Command Query Responsibility Segregation

Separate the write model from the read model so each can be optimised. Full notes: 12-cqrs.md

Must-know

- **Problem** — one model/DB serves complex join reads *and* writes; reads are far more numerous, writes more complex; banking: transactions need accuracy/security, balance checks need speed — **one model can't be tuned for both** (writes want normalised, reads want denormalised pre-joined).
- **How it works** — **Command** changes state and returns only success/failure (or an ID); **Query** returns data and changes nothing. **Never both** → queries are repeatable and cacheable; commands are explicit.
- **Command journey** — validate (format, permissions, rules) → command handler updates state → emit event (`UserAddressUpdated`). **Queries need no validation and emit no events.**
- **Three levels** — (1) separate models, **one DB** (most teams stop here); (2) **separate read/write DBs**, synced by async events → eventual consistency; (3) **event sourcing**: append-only event store, rebuild read models by replay.
- **CQRS ≠ event sourcing ≠ two databases** — independent decisions with very different costs.
- **Consistency window** — user saves, page still shows the old name. Fixes: **read-your-own-writes** (route reads to the write model right after a write), return new state for optimistic UI, version token + poll.

Key facts & numbers

Level	You get	You pay
1 — two models, one DB	Design clarity	Almost nothing
2 — two databases	Independent scaling, tech choice per side	No atomic transactions; out-of-order events, event loss, sync discrepancies, read-model update failures, UI timing
3 — event sourcing	Ordering fix, audit, rebuild/new read models by replay	Big undertaking: event schema evolution, **snapshots**, replay tooling

	Read replicas	CQRS
Model	Same schema copied	Different model shaped for reading

- Multi-command logical transaction failing midway → no rollback → use a **Saga**.
- **When not to use** — simple CRUD entities; parts where read and write needs don't diverge (it adds indirection, pure overhead when workloads don't conflict); never across the whole application by default.
- **Real examples** — e-commerce browsing (millions of reads) vs orders; order lifecycle (placed/paid/shipped/delivered as commands; tracking & notification services query); social media posting (fast, reliable) vs engagement analytics (heavy, not time-critical); collaborative editing (commands can be rejected by rules). Frameworks: **Axon**; plain Spring (two repositories) is legitimate Level 1.

Code to remember

```
// Read-your-own-writes to hide the consistency window
if (recentlyWrittenBy(userId)) return writeModel.find(userId);
return readModel.find(userId);
```

In the interview

Q: Does CQRS need two databases or event sourcing? **A:** No — Level 1 is two models over one DB; event sourcing is a separate, much bigger decision.

Q: Central trade-off? **A:** The consistency window between write and read models — users can see stale data right after saving.

Q: What does event sourcing solve for Level 2? **A:** Out-of-order delivery leaving read models stale — replay the append-only stream to rebuild them; plus audit and new views over history.

Q: CQRS vs read replicas? **A:** Replicas copy the same schema for load; CQRS uses a different, read-shaped model. You can use both.

Gotchas

- Commands that return the updated entity → the rule collapses back into one model.
- Business logic in projections → read and write sides disagree; projections only *shape* data.
- Jumping straight to Level 3 → most common way teams get into trouble.
- 100,000-event aggregates loaded by full replay → use snapshots every N events.

TL;DR

Commands write, queries read, never both; start with two models on one DB; separate DBs and event sourcing are costly upgrades that bring a consistency window.

13. Configuration Externalization

Move config out of code and deployment packages into a central store fetched at runtime. Full notes: 13-config-externalization.md

Must-know

- **Problem** — hardcoded (in source → recompile) or bundled (in the package → rebuild + redeploy) config ties a one-line change to a full deploy, **destroying independent deployability**; can't adapt to changing conditions; secrets scattered across services and Git history.
- **How it works** — config lives **outside the deployment artifact** in a **configuration server** (organised by **environment**, **service**, **instance**); services fetch at **startup, on an interval, or on refresh**, without redeploying.
- **Git-backed store** — versioning, diffs, authorship, code review, **rollback** for free (Spring Cloud Config).
- **Refresh without restart** — `@RefreshScope` beans + `POST /actuator/refresh`; fleet-wide via **Spring Cloud Bus** (`/actuator/busrefresh` over Kafka/RabbitMQ).
- **Secrets** — belong in a **secrets manager** (Vault, AWS Secrets Manager, KMS): leasing, automatic rotation, fine-grained access, audit logs. The pattern **relocates** the secret problem; it doesn't eliminate it.
- **Finding the config server** — via service discovery or a stable Kubernetes Service DNS name (hardcoding its address reintroduces the problem).

Key facts & numbers

Trade-off	Mitigation
Config server = SPOF	≥2 replicas (HA), health checks, **client-side cache of last-known-good** (running services keep running)
New attack surface holding secrets	Encryption in transit, key rotation, trust-boundary separation
Remote fetch latency	Clients cache (Spring Cloud Config caches by default); balance refresh frequency vs freshness
No build/test/review on config changes	Git + pull requests + CI validation of the config repo

Option	Refresh without restart?
Environment variables	**No** — restart required (and no central management/versioning)
K8s ConfigMap as **env vars**	**No**
K8s ConfigMap as **volume**	**Yes**, propagates within ~1 min
Config server + `@RefreshScope`	**Yes**

- Client: `fail-fast: true` **plus** retry (`initial-interval: 1000, max-attempts: 6, multiplier: 1.5`) → wait and retry rather than boot with silently wrong defaults.
- **When not to use** — a config server for secrets (use Vault); a heavy server where simple env vars or ConfigMaps suffice.
- **Real examples** — e-commerce with hundreds of services across dev/staging/prod; regulated finance (no secrets in code or containers — auditors); global fast-food chain (per-country tax, currency, language at boot); streaming startup (feature flags, A/B params, many deploys/day); telecom (bandwidth, routing, security protocols).

Code to remember

```
@RefreshScope                          // bean recreated on /actuator/refresh
@Component
public class CheckoutService {
    @Value("${feature.newCheckout:false}")
    private boolean newCheckoutEnabled;  // NOT static final
}
```

In the interview

Q: What does a Git-backed config store give you? **A:** History, diffs, authorship, review and rollback of every config change.

Q: Isn't the config server a SPOF? **A:** Yes — run it HA and cache last-known-good on clients so an outage stops changes and new starts, not running services.

Q: Secrets in the config server? **A:** Better than hardcoding, but use Vault/Secrets Manager for leasing, rotation and audit; keep ordinary config in Git.

Q: Are config changes low risk? **A:** No — no deploy means no build, tests or review unless you add PRs and CI validation.

Gotchas

- Value read once into a `static final` field → refresh does nothing.
- `encrypt.key` in a plain properties file → the key to all secrets in the open; put it in a KMS/Vault.
- Secrets committed to the config repo → in history forever; rotate, don't just delete.
- Assuming env-var ConfigMaps refresh → only volume mounts do.

TL;DR

Central, Git-backed config fetched at runtime and refreshed without restart; HA + client caching, secrets in Vault, find the server via discovery.

Last-minute checklist

- Circuit breaker: **Closed = flowing, Open = blocked**; Half-Open lets only a few probes through.
- Breaker thresholds: **rate over a sliding window + minimum calls**, slow calls count, 4xx ignored, fallback is local.
- Retry: **transient only**, capped exponential backoff **+ jitter**, few attempts, **overall deadline**, **one layer**.
- Idempotency key generated **once, outside** the retry loop; server dedupes via unique constraint.
- Resilience4j order: **Retry(CircuitBreaker(call))**.
- Bulkhead = **how much**; breaker = **how long**; retry = **how many**. Bulkheads add resilience, **not** capacity.
- Little's Law: **concurrency = throughput × latency** (50 req/s × 0.2 s = 10).
- Bounded queues only — `newFixedThreadPool` has an unbounded queue (OOM risk).
- Discovery: client-side (Eureka + Ribbon) vs server-side (Kubernetes CoreDNS + etcd); **Consul/ZK = CP, Eureka = AP**.
- Register only when ready (readiness probe); deregister before going offline.
- Gateway: HA replicas, parallel fan-out with per-call timeouts, shared-store rate limiting, **transport not domain**.
- BFF: per **client type**, stateless, owned by the frontend team; GraphQL solves shaping, not client logic.
- Sidecar: **separate container, same Pod**, shared network + lifecycle; transparently intercepts traffic; 100 MB × 1,000 pods = 100 GB.
- Saga: local transactions + **compensations** (not rollbacks); **no isolation**; irreversible steps last; outbox.
- Choreography for 2–3 simple steps; **orchestration** beyond that (Temporal, Step Functions, Camunda, Axon).
- 2PC holds locks and blocks if the coordinator dies — why microservices avoid it.
- EDA: events are past tense; **at-least-once** → idempotent consumers; ordering **per partition**; additive schemas only; DLQ; correlation IDs.
- Kafka: `acks: all`, producer idempotence, **manual commit after processing**.
- CQRS: commands return nothing, queries change nothing; **Level 1 (one DB) first**; event sourcing is a separate decision; handle the consistency window (read-your-own-writes).
- Config: Git-backed, `@RefreshScope` + `/actuator/refresh` (Bus for fleet), `fail-fast` + retry, **secrets in Vault**, env-var ConfigMaps don't refresh.
- Strangler Fig: consistent (hashed) routing, shadow mode, deploy the facade first, **commit to a decommission date**.
- Every central component is a SPOF → **HA + cache last-known-good**. Every indirection is a hop.
- Close every answer with the trade-off and the neighbouring pattern.